Ricorsione

Indice

Algoritmi ricorsivi

Albero delle chiamate ricorsive e call stack

Correttezza degli algoritmi ricorsivi

Fattoriale

Numeri di Fibonacci

Massimo comun divisore

Elevamento a potenza veloce

Torri di Hanoi

Dagli algoritmi iterativi agli algoritmi ricorsivi

- Nei capitoli precedenti ogni algoritmo era **iterativo**: una sequenza di istruzioni e di cicli
- Ne abbiamo contato i passi riga per riga (capitolo «Tempo di esecuzione») e ne abbiamo dimostrato la correttezza con gli invarianti di ciclo (capitolo «Correttezza»)

In questo capitolo studiamo un modo diverso di progettare gli algoritmi:
un algoritmo può **richiamare se stesso**
su istanze più piccole dello stesso problema.

Il piano del capitolo

- Definiamo gli algoritmi ricorsivi, i casi base e i passi ricorsivi, e vediamo perché un algoritmo ricorsivo termina
- Descriviamo come viene eseguito un algoritmo ricorsivo: l'**albero delle chiamate ricorsive** (che determina il numero di passi) e il **call stack** (che determina la memoria)
- Mostriamo che la correttezza di un algoritmo ricorsivo si dimostra per **induzione forte**
- Applichiamo questi strumenti a cinque classici algoritmi ricorsivi: il fattoriale, i numeri di Fibonacci, il massimo comun divisore, l'elevamento a potenza veloce e le Torri di Hanoi
- Per ciascuno di essi dimostriamo la correttezza, contiamo il numero di passi srotolando un'equazione di ricorrenza e lo eseguiamo passo per passo

Algoritmi ricorsivi

Introduzione: Algoritmi ricorsivi

- In questa sezione definiamo che cos'è un algoritmo ricorsivo: casi base e passo ricorsivo
- Vediamo perché un algoritmo ricorsivo **termina**, e che cosa può andare storto quando non termina
- Infine fissiamo come si contano i passi di un algoritmo ricorsivo: si arriva a un'**equazione di ricorrenza**

Problemi autosimili

- Molti problemi hanno una struttura **autosimile** naturale: la soluzione di un'istanza si ottiene facilmente dalla soluzione di una o più **istanze più piccole dello stesso problema**

- Per esempio, per ogni numero naturale $n \geq 1$,

$$n! = n \cdot (n-1)!$$

  cioè il fattoriale di n si ottiene dal fattoriale del numero più piccolo $n-1$

- Gli algoritmi che sfruttano questa struttura si chiamano algoritmi ricorsivi

Definizione 1: algoritmo ricorsivo

*Un **algoritmo ricorsivo** è un algoritmo che risolve un'istanza di un problema **richiamando se stesso** su una o più **istanze più piccole** dello stesso problema, e combinando poi i risultati di queste chiamate.*

Le due parti di un algoritmo ricorsivo

Ogni algoritmo ricorsivo è composto da due parti:

1. **Caso/i base**: una o più istanze abbastanza piccole da essere risolte direttamente, senza alcuna chiamata ricorsiva

   (per esempio, $0! = 1$)

2. **Passo ricorsivo**: su ogni altra istanza, l'algoritmo richiama se stesso su istanze più piccole e combina i risultati restituiti

   (per esempio, $n! = n \cdot (n-1)!$)

Chiamate e dimensione

- Ogni esecuzione dell'algoritmo su un dato input si chiama **chiamata** (o invocazione)
- Una chiamata effettuata dall'interno di un'altra chiamata è una **chiamata ricorsiva**
- Per misurare quanto sia "piccola" un'istanza, associamo a ogni input una **dimensione**, che è un numero naturale
- Per esempio il valore n del numero in input, oppure il numero n di elementi di un array $a[1..n]$

Terminazione

- Un algoritmo ricorsivo termina soltanto se la catena delle chiamate ricorsive raggiunge prima o poi un caso base
- Se la dimensione dell'istanza diminuisce strettamente a ogni chiamata ricorsiva, otteniamo una successione strettamente decrescente di numeri naturali

$$n_0 > n_1 > n_2 > \cdots$$

- Non esiste alcuna successione infinita strettamente decrescente di numeri naturali (per la proprietà del buon ordinamento di $\mathbb{N}$): dopo al più n_0 chiamate la dimensione non può più diminuire

Terminazione

Quindi, se ogni istanza che non è un caso base viene ridotta a istanze strettamente più piccole,

un caso base viene necessariamente raggiunto dopo un numero finito di chiamate.

Idea chiave della ricorsione

- Risolvere direttamente i **casi base**
- Ridurre ogni altra istanza a una o più istanze **strettamente più piccole** dello stesso problema, risolverle con chiamate ricorsive e combinare i loro risultati
- Poiché la dimensione è un numero naturale che **diminuisce strettamente** a ogni chiamata ricorsiva, un caso base viene sempre raggiunto e l'algoritmo **termina**

Che cosa succede se manca uno di questi ingredienti?
Vediamo che cosa può andare storto.

Esempio: che cosa può andare storto (1/2)

Caso base mancante.

- Consideriamo la funzione `WrongFactorial`(n) che restituisce $n \cdot$ `WrongFactorial`$(n-1)$, senza alcun caso base
- La chiamata `WrongFactorial`(2) richiama `WrongFactorial`(1),

  che richiama `WrongFactorial`(0),

  che richiama `WrongFactorial`(-1), e così via all'infinito
- La ricorsione non si ferma mai

Esempio: che cosa può andare storto (2/2)

Dimensione che non diminuisce.

- Una funzione che, sull'input n, richiama se stessa sull'input n (oppure $n+1$) non raggiunge mai un caso base $n = 0$ quando $n \geq 1$

In pratica.

- Una ricorsione infinita viene fermata dal calcolatore quando la memoria riservata alle chiamate si esaurisce
- È un errore di **stack overflow**

Contare i passi di un algoritmo ricorsivo

- Usiamo le stesse convenzioni del capitolo «Tempo di esecuzione»: ogni esecuzione di una riga di pseudocodice è **un passo**
- Una riga che contiene una chiamata ricorsiva conta come un passo **più** tutti i passi eseguiti da quella chiamata ricorsiva
- Di conseguenza, il numero di passi $S(n)$ di un algoritmo ricorsivo si esprime naturalmente in termini del numero di passi su istanze più piccole, cioè mediante un'**equazione di ricorrenza**, come

$$S(n) = S(n-1) + 2.$$

Risolvere le equazioni di ricorrenza

In questo capitolo risolviamo le equazioni di ricorrenza dei conteggi dei passi per **semplice srotolamento** (detto anche **espansione**):

sostituiamo ripetutamente $S(\cdot)$ con la sua definizione fino a raggiungere un caso base.

Riepilogo

Algoritmi ricorsivi:

- richiamano se stessi su istanze più piccole dello stesso problema e combinano i risultati
- caso/i base, risolti direttamente, e passo ricorsivo

Terminazione:

- la dimensione è un numero naturale che diminuisce strettamente a ogni chiamata ricorsiva
- senza caso base, o con una dimensione che non diminuisce: ricorsione infinita (stack overflow)

Numero di passi:

- descritto da un'equazione di ricorrenza, risolta per srotolamento

Albero delle chiamate ricorsive e call stack

Introduzione: Albero delle chiamate ricorsive e call stack

- In questa sezione descriviamo come viene eseguito, in concreto, un algoritmo ricorsivo
- L'**albero delle chiamate ricorsive** mostra tutte le chiamate: il suo numero di nodi determina il numero di passi
- Il **call stack** conserva in memoria le chiamate attive: la sua dimensione è data dalla **profondità** dell'albero
- Useremo queste due immagini per ogni algoritmo del capitolo

Definizione 2: albero delle chiamate ricorsive

*L'**albero delle chiamate ricorsive** di un algoritmo ricorsivo su un dato input è l'albero radicato in cui:*

- *ogni nodo è una chiamata dell'algoritmo;*
- *la radice è la chiamata iniziale;*
- *i figli di un nodo sono le chiamate ricorsive effettuate direttamente da quella chiamata, da sinistra a destra nell'ordine in cui vengono eseguite.*

Le foglie dell'albero sono le chiamate che non effettuano chiamate ricorsive (tipicamente i casi base).

Nodi, passi e ordine delle chiamate

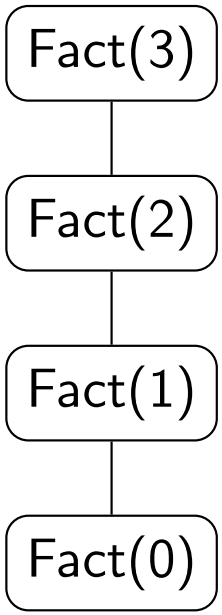
- Il numero totale di chiamate eseguite dall'algoritmo è il **numero di nodi** dell'albero
- Se ogni chiamata esegue un numero costante di passi (esclusi i passi delle sue chiamate ricorsive), allora il numero di passi dell'algoritmo è proporzionale al numero di nodi dell'albero
- Le chiamate vengono eseguite seguendo una visita **in profondità** dell'albero:
  - una chiamata inizia, esegue completamente la sua prima chiamata ricorsiva (e l'intero sottoalbero sottostante), poi la seconda, e così via;
  - restituisce il suo risultato solo quando tutte le sue chiamate ricorsive sono terminate.

Esempio: due forme di alberi delle chiamate ricorsive (1/2)

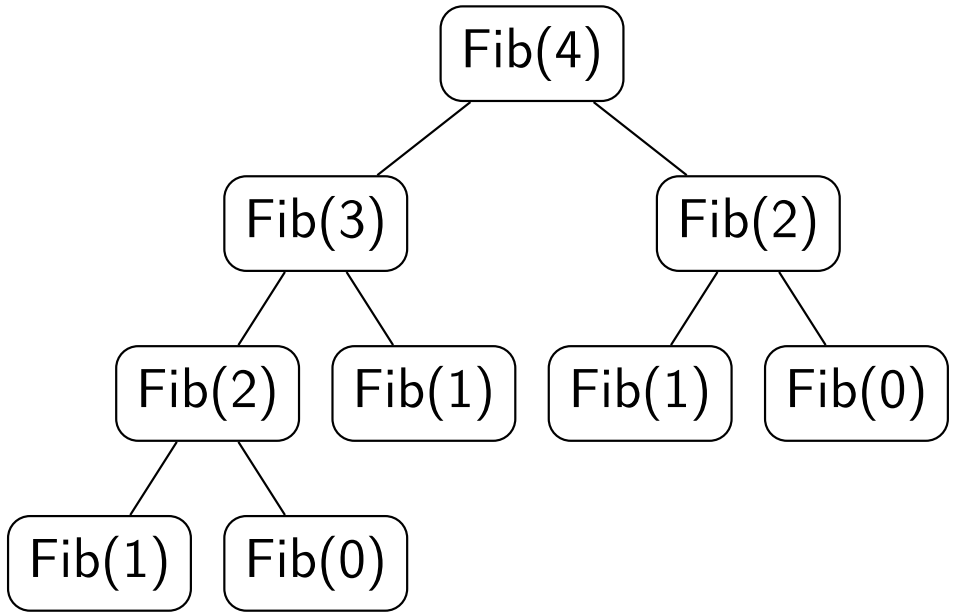
- Se ogni chiamata effettua **una** sola chiamata ricorsiva (come per $n! = n \cdot (n-1)!$), l'albero è un **cammino**
- Se ogni chiamata effettua **due** chiamate ricorsive, l'albero si **ramifica** e il numero di nodi può crescere molto rapidamente
- Succede per i numeri di Fibonacci, definiti da

$$F(n) = F(n-1) + F(n-2) \qquad \text{con } F(0) = 0 \text{ e } F(1) = 1$$

Esempio: due forme di alberi delle chiamate ricorsive (2/2)



una chiamata: un cammino



due chiamate: l'albero si ramifica

Stack frame

- Quando una chiamata inizia, il calcolatore riserva un pezzo di memoria, detto **stack frame** (o record di attivazione)
- Il frame contiene:
  - i parametri della chiamata;
  - le sue variabili locali;
  - il punto del programma in cui l'esecuzione deve proseguire quando la chiamata termina.

Il call stack

- I frame sono memorizzati in un'area di memoria organizzata come una **pila** (lo **stack** delle chiamate, o **call stack**):
  - quando una chiamata inizia, il suo frame viene inserito (**push**) in cima alla pila;
  - quando una chiamata termina, il suo frame viene rimosso (**pop**) dalla cima della pila, e l'esecuzione prosegue nella chiamata il cui frame è ora in cima.
- In ogni momento, i frame nella pila sono esattamente le chiamate sul cammino dalla radice dell'albero delle chiamate ricorsive fino alla chiamata attualmente in esecuzione

Definizione 3: profondità della ricorsione

*La **profondità della ricorsione** di un algoritmo ricorsivo su un dato input è il massimo numero di chiamate simultaneamente attive, cioè il massimo numero di nodi su un cammino dalla radice a una foglia dell'albero delle chiamate ricorsive.*

Memoria usata dal call stack

Il massimo numero di stack frame
simultaneamente in memoria
è uguale alla **profondità della ricorsione**.

Se ogni frame usa una quantità costante di memoria,
la memoria usata dal call stack è

Θ(profondità della ricorsione).

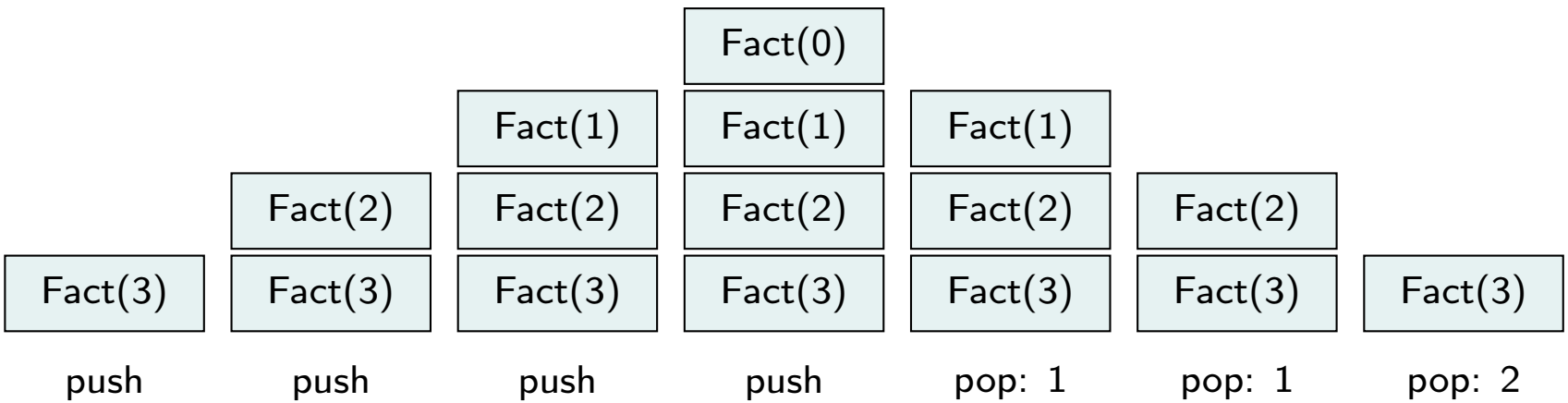
Passi e memoria

- Si noti la differenza:
  - il **numero di passi** dipende dal **numero di nodi** dell'albero;
  - la **memoria** usata dal call stack dipende solo dalla sua **profondità**.

- Per l'albero di `Fib(4)` mostrato sopra:
  - ci sono 9 chiamate;
  - ma al più 4 di esse sono contemporaneamente nella pila (per esempio `Fib(4)`, `Fib(3)`, `Fib(2)`, `Fib(1)`).

Esempio: evoluzione del call stack per Fact(3) (1/2)

La figura mostra il call stack (la cima della pila è in alto) dopo ogni push e dopo ogni pop, durante il calcolo di 3! con la ricorsione $n! = n \cdot (n-1)!$, $0! = 1$.



Esempio: evoluzione del call stack per Fact(3) (2/2)

- Vengono inseriti quattro frame: `Fact(3)`, `Fact(2)`, `Fact(1)`, `Fact(0)`
- `Fact(0)` è un caso base: restituisce 1 e il suo frame viene rimosso
- Poi `Fact(1)` restituisce $1 \cdot 1 = 1$, `Fact(2)` restituisce $2 \cdot 1 = 2$ e, infine, `Fact(3)` restituisce $3 \cdot 2 = 6$ e la pila diventa vuota
- La profondità della ricorsione è 4

La dimensione del call stack è limitata

La dimensione del call stack è limitata
(tipicamente alcuni megabyte).

Una ricorsione molto profonda
(per esempio, milioni di chiamate annidate)
causa uno **stack overflow**,
anche se l'algoritmo è corretto e termina.

Riepilogo

Albero delle chiamate ricorsive:

- nodi = chiamate, figli = chiamate ricorsive nell'ordine di esecuzione
- numero di nodi = numero di chiamate; le chiamate seguono una visita in profondità

Call stack:

- un frame viene inserito quando una chiamata inizia, rimosso quando termina
- i frame nella pila sono le chiamate sul cammino dalla radice alla chiamata corrente

Passi e memoria:

- passi $\leftrightarrow$ numero di nodi; memoria $\leftrightarrow$ profondità della ricorsione

Correttezza degli algoritmi ricorsivi

Introduzione: Correttezza degli algoritmi ricorsivi

- In questa sezione vediamo come si dimostra che un algoritmo ricorsivo restituisce l'output corretto
- Lo strumento è l'**induzione forte** sulla dimensione dell'input: la struttura di un algoritmo ricorsivo rispecchia la struttura di una dimostrazione per induzione
- Confrontiamo anche questo approccio con gli invarianti di ciclo usati per gli algoritmi iterativi

Induzione sulla dimensione dell'input

- Lo strumento naturale per dimostrare la correttezza di un algoritmo ricorsivo è l'**induzione matematica** sulla dimensione dell'input
- Una chiamata ricorsiva può essere effettuata su una qualunque istanza più piccola, non soltanto sull'istanza di dimensione $n - 1$
- Per questo usiamo l'**induzione forte**

Induzione forte

Sia $P(n)$ un'affermazione che dipende da $n \in \mathbb{N}$, e sia $n_0 \in \mathbb{N}$. Se

1. **caso base**: $P(n_0)$ è vera, e
2. **passo induttivo**: per ogni $n > n_0$, se $P(k)$ è vera per ogni k con $n_0 \leq k < n$ (**ipotesi induttiva**), allora $P(n)$ è vera,

allora $P(n)$ è vera per ogni numero naturale $n \geq n_0$.

- A volte conviene dimostrare più di un caso base (per esempio $n = 0$ e $n = 1$), e poi il passo induttivo solo per n maggiore di tutti i casi base

La ricorsione rispecchia l'induzione

La struttura di un algoritmo ricorsivo rispecchia la struttura di una dimostrazione per induzione:

- i **casi base** dell'algoritmo corrispondono ai **casi base** dell'induzione;
- nel **passo ricorsivo** possiamo assumere (ipotesi induttiva) che le chiamate ricorsive, effettuate su input più piccoli, restituiscano risultati corretti;
- dobbiamo soltanto mostrare che i risultati vengano **combinati** correttamente.

Osservazione 1: correttezza degli algoritmi ricorsivi

Consideriamo un algoritmo ricorsivo in cui ogni input ha una dimensione in $\mathbb{N}$. Supponiamo che:

1. *su ogni input che è un caso base, l'algoritmo restituisca l'output corretto;*
2. *su ogni altro input di dimensione n, l'algoritmo effettui un numero finito di chiamate ricorsive, tutte su input di dimensione strettamente minore di n, e restituisca l'output corretto ogniqualvolta tutte queste chiamate ricorsive restituiscono l'output corretto.*

Allora l'algoritmo termina e restituisce l'output corretto su ogni input.

Dimostrazione (1/2)

Sia $P(n)$ l'affermazione:

"su ogni input di dimensione n l'algoritmo termina e restituisce l'output corretto".

Dimostriamo $P(n)$ per ogni $n \in \mathbb{N}$ per induzione forte.

- Sia $n \in \mathbb{N}$ e supponiamo che $P(k)$ sia vera per ogni $k < n$

  (per $n = 0$ questa ipotesi è vuota)

- Consideriamo un input di dimensione n

Dimostrazione (2/2)

- Se l'input è un caso base, allora l'algoritmo termina e restituisce l'output corretto per l'ipotesi 1
- Altrimenti, per l'ipotesi 2, l'algoritmo effettua un numero finito di chiamate ricorsive su input di dimensione $k < n$
  - per l'ipotesi induttiva, ciascuna di queste chiamate termina e restituisce l'output corretto;
  - quindi l'algoritmo termina e, per l'ipotesi 2, restituisce l'output corretto.

Quindi $P(n)$ è vera, e per induzione forte $P(n)$ è vera per ogni $n \in \mathbb{N}$. □

Legame con gli invarianti di ciclo

- Nel capitolo «Correttezza» abbiamo dimostrato la correttezza degli algoritmi iterativi per mezzo degli **invarianti di ciclo**, che hanno anch'essi la struttura di una dimostrazione per induzione
- Le tre proprietà di un invariante di ciclo corrispondono alle parti della dimostrazione per induzione di un algoritmo ricorsivo

Invarianti di ciclo e induzione

Invariante di ciclo (algoritmo iterativo)	**Induzione** (algoritmo ricorsivo)
Inizializzazione: vero prima della prima iterazione	Caso base (o casi base) della ricorsione
Conservazione: preservato da ogni iterazione	Passo induttivo: corretta combinazione dei risultati
Terminazione: il ciclo si ferma	La dimensione diminuisce strettamente verso un caso base

Due direzioni

La differenza principale è la direzione.

Un algoritmo iterativo costruisce la soluzione "dal basso"
(dall'inizializzazione verso l'ultima iterazione).

Un algoritmo ricorsivo parte "dall'alto"
(l'istanza originale) e la riduce a istanze più piccole
fino a raggiungere un caso base.

Riepilogo

Induzione forte:

- l'ipotesi induttiva vale per ogni dimensione più piccola, non solo per $n - 1$
- eventualmente con più di un caso base

Correttezza di un algoritmo ricorsivo:

- output corretto sui casi base
- un numero finito di chiamate ricorsive su input strettamente più piccoli, i cui risultati corretti vengono combinati correttamente

Invarianti di ciclo:

- inizializzazione $\leftrightarrow$ casi base, conservazione $\leftrightarrow$ passo induttivo, terminazione $\leftrightarrow$ dimensione che diminuisce

Fattoriale

Introduzione: Fattoriale

- In questa sezione scriviamo l'algoritmo ricorsivo più semplice: il calcolo di $n!$
- Ne disegniamo l'albero delle chiamate ricorsive, ne dimostriamo la correttezza per induzione e ne contiamo i passi srotolando l'equazione di ricorrenza $S(n) = S(n-1) + 2$
- Esprimiamo anche il suo tempo di esecuzione in funzione della dimensione dell'input, e lo eseguiamo passo per passo

Il problema

Dato un numero naturale n, consideriamo il problema di calcolare il **fattoriale** di n:

$$n! = 1 \cdot 2 \cdot \ldots \cdot n \qquad \text{con } 0! = 1.$$

- Poiché $n! = n \cdot (n-1)!$ per ogni $n \geq 1$, il fattoriale ha la seguente definizione ricorsiva:

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \cdot (n-1)! & \text{se } n \geq 1 \end{cases}$$

Questo è lo pseudocodice dell'algoritmo ricorsivo `Factorial`.

Algorithm: Factorial

Input : Un numero naturale n
Output : Il fattoriale $n!$

```
1 function Factorial(n)
2 |   if n = 0 then
3 |   |   return 1                          // caso base
4 |   end
5 |   return n · Factorial(n − 1)          // passo ricorsivo
6 end
```

Esegui in Colab: Python · C++ · Java

Come funziona Factorial

- Il caso base è $n = 0$, per il quale l'algoritmo restituisce $0! = 1$
- Per $n \geq 1$, l'algoritmo richiama se stesso sull'input più piccolo $n - 1$ e moltiplica il risultato per n
- La dimensione dell'input è il valore n, che diminuisce di uno a ogni chiamata ricorsiva: il caso base viene raggiunto dopo n chiamate ricorsive
- In un'implementazione in C++ con il tipo `unsigned long long` (64 bit), il risultato è corretto solo per $n \leq 20$, poiché $21! > 2^{64} - 1$

Esempio: albero delle chiamate ricorsive di Factorial(4)

L'albero è un cammino con 5 nodi. Il valore restituito da ciascuna chiamata è scritto in rosso accanto alla chiamata.



Le chiamate vengono effettuate dall'alto verso il basso; i risultati vengono restituiti dal basso verso l'alto. La profondità della ricorsione è 5.

Osservazione 2: correttezza di Factorial

Per ogni numero naturale n, la chiamata `Factorial`(n) *termina e restituisce* $n!$.

Dimostrazione

Procediamo per induzione su n.

- **Caso base** ($n = 0$): la condizione della riga 2 è vera e l'algoritmo restituisce $1 = 0!$ alla riga 3
- **Passo induttivo** ($n \geq 1$): supponiamo che $\texttt{Factorial}(n-1)$ termini e restituisca $(n-1)!$ (ipotesi induttiva). Poiché $n \neq 0$, l'algoritmo esegue la riga 5 e restituisce

$$n \cdot \texttt{Factorial}(n-1) = n \cdot (n-1)! = n!.$$

Quindi $\texttt{Factorial}(n)$ restituisce $n!$ per ogni $n \in \mathbb{N}$. □

Numero di passi: l'equazione di ricorrenza

- Sia $S(n)$ il numero di passi eseguiti da `Factorial` sull'input n (per ogni valore di n c'è un solo input, quindi questo è anche il caso peggiore)

- Se $n = 0$, le righe 2 e 3 vengono eseguite una volta ciascuna:

$$S(0) = 2.$$

- Se $n \geq 1$, le righe 2 e 5 vengono eseguite una volta ciascuna, e la riga 5 esegue anche tutti i passi della chiamata `Factorial`$(n-1)$:

$$S(n) = S(n-1) + 2.$$

Numero di passi: srotolare l'equazione di ricorrenza

Risolviamo l'equazione di ricorrenza srotolandola:

$$\begin{aligned} S(n) &= S(n-1) + 2 \\ &= \big(S(n-2) + 2\big) + 2 = S(n-2) + 2 \cdot 2 \\ &= \big(S(n-3) + 2\big) + 2 \cdot 2 = S(n-3) + 3 \cdot 2 \\ &\vdots \\ &= S(n-k) + k \cdot 2 \qquad \forall k \in \{1, 2, \ldots, n\}. \end{aligned}$$

Scegliendo $k = n$ (caso base), otteniamo

$$S(n) = S(0) + 2n = 2n + 2.$$

Numero di passi e profondità

- Quindi,

$$S(n) = 2n + 2 = \Theta(n),$$

poiché

$$\lim_{n\to\infty} \frac{2n+2}{n} = \lim_{n\to\infty} \left(2 + \frac{2}{n}\right) = 2 \in \mathbb{R}_{>0}.$$

- La profondità della ricorsione è $n + 1$ (le chiamate con input $n, n-1, \ldots, 0$ sono tutte contemporaneamente nella pila), quindi il call stack usa una memoria $\Theta(n)$

- L'algoritmo iterativo che calcola $n!$ con un ciclo `while` usa soltanto una quantità costante di memoria

Esprimiamo ora il numero di passi
in funzione della dimensione dell'input $|I|$.

Passi in funzione della dimensione dell'input (1/2)

- Come nel capitolo «Tempo di esecuzione», contiamo ogni moltiplicazione come un passo
- L'input è il numero naturale n, che richiede $\lfloor \log_2 n \rfloor + 1$ bit (per $n \geq 1$), quindi

$$|I| = \Theta(\log_2 n).$$

- Questo significa che esistono costanti $c_1, c_2 > 0$ tali che, per ogni n sufficientemente grande,

$$c_1 \log_2 n \leq |I| \leq c_2 \log_2 n \qquad \Longleftrightarrow \qquad 2^{\frac{1}{c_2}|I|} \leq n \leq 2^{\frac{1}{c_1}|I|}.$$

Passi in funzione della dimensione dell'input (2/2)

- Poiché $S(n) = \Theta(n)$, esistono costanti $a_1, a_2 > 0$ tali che

$$a_1 n \leq S(n) \leq a_2 n$$

per ogni n sufficientemente grande

- Quindi

$$a_1 2^{\frac{1}{c_2}|I|} \leq S(n) \leq a_2 2^{\frac{1}{c_1}|I|},$$

cioè

$$T(|I|) = 2^{\Theta(|I|)}.$$

Tempo di esecuzione di Factorial

- Il numero di passi dell'algoritmo `Factorial` è

$$S(n) = 2n + 2 = \Theta(n),$$

e la sua profondità di ricorsione è $n + 1$

- In funzione della dimensione dell'input $|I| = \Theta(\log_2 n)$, il tempo di esecuzione nel caso peggiore è

$$T(|I|) = 2^{\Theta(|I|)},$$

cioè è **lineare** nel valore numerico n,
ma **esponenziale** nella dimensione dell'input $|I|$.

Esempio: esecuzione di Factorial(4) (1/2)

Chiamate (lo stack cresce)

1. `Factorial(4)`: $4 \neq 0 \Rightarrow$ deve calcolare $4 \cdot$ `Factorial(3)` e attende. stack: [4]
2. `Factorial(3)`: $3 \neq 0 \Rightarrow$ deve calcolare $3 \cdot$ `Factorial(2)` e attende. stack: [4, 3]
3. `Factorial(2)`: $2 \neq 0 \Rightarrow$ deve calcolare $2 \cdot$ `Factorial(1)` e attende. stack: [4, 3, 2]
4. `Factorial(1)`: $1 \neq 0 \Rightarrow$ deve calcolare $1 \cdot$ `Factorial(0)` e attende. stack: [4, 3, 2, 1]
5. `Factorial(0)`: $n = 0 \Rightarrow$ caso base. stack: [4, 3, 2, 1, 0]

Esempio: esecuzione di Factorial(4) (2/2)

Ritorni (lo stack si riduce)

1. `Factorial(0)` restituisce 1. stack: [4, 3, 2, 1]
2. `Factorial(1)` restituisce $1 \cdot 1 = 1$. stack: [4, 3, 2]
3. `Factorial(2)` restituisce $2 \cdot 1 = 2$. stack: [4, 3]
4. `Factorial(3)` restituisce $3 \cdot 2 = 6$. stack: [4]
5. `Factorial(4)` restituisce $4 \cdot 6 = 24$. stack: [ ]

L'algoritmo termina con risultato $24 = 4!$, dopo 5 chiamate.

Riepilogo

L'algoritmo:

- caso base $n = 0$, passo ricorsivo $n \cdot \texttt{Factorial}(n-1)$
- l'albero delle chiamate ricorsive è un cammino con $n+1$ nodi

Correttezza:

- per induzione su n

Numero di passi:

- $S(n) = S(n-1) + 2$, quindi $S(n) = 2n + 2 = \Theta(n)$; profondità $n+1$
- $T(|I|) = 2^{\Theta(|I|)}$: esponenziale nella dimensione dell'input

Numeri di Fibonacci

Introduzione: Numeri di Fibonacci

- In questa sezione l'algoritmo ricorsivo effettua **due** chiamate ricorsive, e il suo albero delle chiamate ricorsive si ramifica
- Dimostriamo che il numero di chiamate cresce **esponenzialmente**, perché gli stessi sottoproblemi vengono risolti più e più volte
- Confrontiamo l'algoritmo ricorsivo ingenuo con un algoritmo iterativo che esegue soltanto un numero lineare di passi

Il problema

Dato un numero naturale n, consideriamo il problema di calcolare l'n-esimo **numero di Fibonacci** $F(n)$, definito come

$$F(n) = \begin{cases} n & \text{se } n \in \{0, 1\} \\ F(n-1) + F(n-2) & \text{se } n \geq 2 \end{cases}$$

I primi numeri di Fibonacci sono:

n	0	1	2	3	4	5	6	7	8	9	10
$F(n)$	0	1	1	2	3	5	8	13	21	34	55

La definizione è essa stessa ricorsiva, e fornisce direttamente
il seguente algoritmo ricorsivo (ingenuo) `Fib`,
con **due** casi base e **due** chiamate ricorsive.

Algorithm: Fib

Input : Un numero naturale n
Output : Il numero di Fibonacci $F(n)$

```
1 function Fib(n)
2 |   if n ≤ 1 then
3 |   |   return n                                  // casi base
4 |   end
5 |   return Fib(n − 1) + Fib(n − 2)     // passo ricorsivo
6 end
```

Esegui in Colab: Python · C++ · Java

Osservazione 3: correttezza di Fib

Per ogni numero naturale n, la chiamata `Fib`(n) *termina e restituisce* $F(n)$.

Dimostrazione

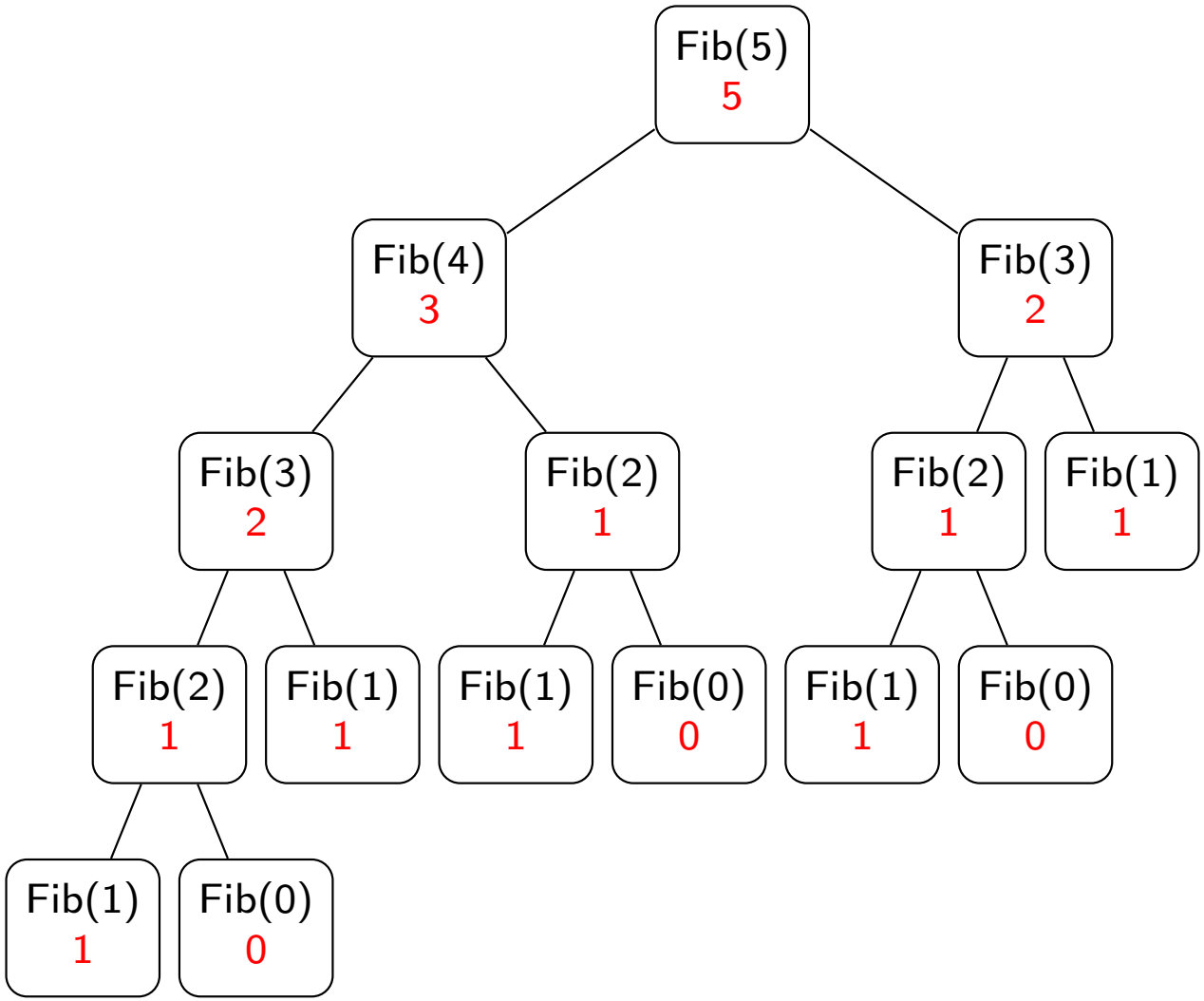
Procediamo per induzione forte su n.

- **Casi base** ($n \in \{0, 1\}$): l'algoritmo restituisce $n = F(n)$ alla riga 3
- **Passo induttivo** ($n \geq 2$): per ipotesi induttiva le due chiamate sugli input più piccoli $n - 1$ e $n - 2$ restituiscono $F(n-1)$ e $F(n-2)$, e la riga 5 restituisce la loro somma $F(n)$

Qui servono due casi base, perché il passo induttivo per n usa sia $n - 1$ sia $n - 2$. □

Esempio: albero delle chiamate ricorsive di Fib(5) (1/2)

Ogni nodo è una chiamata; il valore restituito dalla chiamata è scritto in rosso sotto di essa.



Esempio: albero delle chiamate ricorsive di Fib(5) (2/2)

- L'albero ha 15 nodi: l'algoritmo effettua 15 chiamate per calcolare $F(5) = 5$
- Gli stessi sottoproblemi vengono risolti molte volte:
  - `Fib(3)` viene calcolato 2 volte;
  - `Fib(2)` 3 volte;
  - `Fib(1)` 5 volte;
  - `Fib(0)` 3 volte.
- La profondità della ricorsione è 5 (per esempio, il cammino `Fib(5)`, `Fib(4)`, `Fib(3)`, `Fib(2)`, `Fib(1)`)

Numero di chiamate

- Sia $C(n)$ il numero di chiamate (nodi dell'albero) eseguite da `Fib`(n)
- Ogni chiamata con $n \geq 2$ genera una chiamata su $n-1$ e una chiamata su $n-2$, quindi

$$C(n) = \begin{cases} 1 & \text{se } n \in \{0,1\} \\ C(n-1) + C(n-2) + 1 & \text{se } n \geq 2 \end{cases}$$

- I primi valori sono

$$C(0) = 1, \quad C(1) = 1, \quad C(2) = 3, \quad C(3) = 5,$$
$$C(4) = 9, \quad C(5) = 15, \quad C(6) = 25$$

- Sono legati ai numeri di Fibonacci stessi

Osservazione 4: numero di chiamate di Fib

Per ogni numero naturale n,

$$C(n) = 2F(n+1) - 1.$$

Dimostrazione

Procediamo per induzione forte su n.

- **Casi base**: $C(0) = 1 = 2 \cdot 1 - 1 = 2F(1) - 1$ e $C(1) = 1 = 2 \cdot 1 - 1 = 2F(2) - 1$
- **Passo induttivo** ($n \geq 2$): supponiamo $C(k) = 2F(k+1) - 1$ per ogni $k < n$. Allora

$$\begin{aligned} C(n) &= C(n-1) + C(n-2) + 1 \\ &= \big(2F(n) - 1\big) + \big(2F(n-1) - 1\big) + 1 \\ &= 2\big(F(n) + F(n-1)\big) - 1 = 2F(n+1) - 1. \end{aligned}$$

□

Numero di passi

- Per esempio, $C(5) = 2F(6) - 1 = 2 \cdot 8 - 1 = 15$, come nell'albero di `Fib(5)`
- Ogni chiamata esegue esattamente 2 righe (le righe 2–3 per un caso base, le righe 2 e 5 altrimenti), quindi il numero di passi $S(n)$ soddisfa

  $$S(n) = S(n-1)+S(n-2)+2 \quad (n \geq 2), \qquad S(0) = S(1) = 2,$$

  e perciò

  $$S(n) = 2C(n) = 4F(n+1) - 2.$$

- Il numero di chiamate cresce **esponenzialmente** in n

Osservazione 5: numero esponenziale di chiamate di Fib

Per ogni numero naturale $n \geq 2$,

$$2^{n/2} \leq C(n) \leq 2^n.$$

Dimostrazione (1/2)

Limite inferiore. Procediamo per induzione forte su $n \geq 2$.

- **Casi base**: $C(2) = 3 \geq 2 = 2^{2/2}$ e $C(3) = 5 \geq 2\sqrt{2} = 2^{3/2}$
- **Passo induttivo** ($n \geq 4$): supponiamo l'affermazione vera per ogni k con $2 \leq k < n$; in particolare per $k = n - 2 \geq 2$
- Poiché $C(n-1) \geq C(n-2)$ (la successione C è non decrescente),

$$C(n) = C(n-1) + C(n-2) + 1 > 2\,C(n-2) \geq 2 \cdot 2^{(n-2)/2} = 2^{n/2}.$$

Dimostrazione (2/2)

Limite superiore. Anche per induzione forte:

- $C(2) = 3 \leq 4$ e $C(3) = 5 \leq 8$
- per $n \geq 4$,

  $$C(n) = C(n-1) + C(n-2) + 1 \leq 2^{n-1} + 2^{n-2} + 2^{n-2} = 2^n,$$

  dove abbiamo usato $1 \leq 2^{n-2}$.

□

L'algoritmo ingenuo è esponenziale

- Si noti che $C(1) = 1 < 2^{1/2}$: per questo il limite inferiore è enunciato per $n \geq 2$
- Quindi $S(n) = 2C(n) = 2^{\Theta(n)}$: l'algoritmo ricorsivo ingenuo è **esponenziale** in n
- Per esempio, il calcolo di $F(30) = 832040$ richiede

  $$C(30) = 2F(31) - 1 = 2 \cdot 1346269 - 1 = 2692537 \text{ chiamate}$$

- (Si può dimostrare che $F(n)$ cresce come $\varphi^n/\sqrt{5}$, dove $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ è la sezione aurea; quindi $C(n) = \Theta(\varphi^n)$.)

Un algoritmo iterativo

La ragione del numero esponenziale di chiamate
è che l'algoritmo ingenuo **risolve più e più volte**
gli stessi sottoproblemi (si veda l'albero di `Fib(5)`).

Un algoritmo iterativo calcola ciascun numero di Fibonacci
una sola volta, a partire da $F(0)$ e $F(1)$ in su,
conservando soltanto gli ultimi due valori.

Algorithm: IterativeFib (1/2)

Input : Un numero naturale n
Output : Il numero di Fibonacci $F(n)$

```
1 function IterativeFib(n)
2     if n ≤ 1 then
3         return n
4     end
5     prev := 0
6     curr := 1
7     i := 2
```

Algorithm: IterativeFib (2/2)

```
function IterativeFib(n) (continua)
8       while i ≤ n do
9           next := prev + curr
10          prev ← curr
11          curr ← next
12          i ← i + 1
13      end
14      return curr
15  end
```

Esegui in Colab: Python · C++ · Java

Numero di passi di IterativeFib

- Per $n \geq 2$:
  - le righe 2, 5, 6, 7, 14 vengono eseguite una volta;
  - la riga 8 viene eseguita n volte;
  - le righe 9–12 vengono eseguite $n - 1$ volte.
- Quindi

$$S(n) = 5 + n + 4(n - 1) = 5n + 1 = \Theta(n).$$

- (Invariante di ciclo: prima di ogni verifica della riga 8, $prev = F(i - 2)$ e $curr = F(i - 1)$; alla fine $i = n + 1$ e $curr = F(n)$.)

Ricorsivo e iterativo a confronto

n	$F(n)$	passi di `Fib`: $4F(n+1)-2$	passi di `IterativeFib`: $5n+1$
5	5	30	26
10	55	354	51
20	6765	43782	101
30	832040	5385074	151

- L'algoritmo ricorsivo ingenuo `Fib` esegue $S(n) = 4F(n+1) - 2 = 2^{\Theta(n)}$ passi (esponenziale in n)
- `IterativeFib` esegue $S(n) = 5n + 1 = \Theta(n)$ passi (lineare in n)

In funzione della dimensione dell'input

Poiché $|I| = \Theta(\log_2 n)$,
in funzione della dimensione dell'input:

l'algoritmo ingenuo è addirittura **doppiamente esponenziale**,

mentre quello iterativo è **esponenziale**
(come `Factorial`).

Sottoproblemi ripetuti

- La ricorsione in sé non è il problema
- Il problema è la **ripetizione di sottoproblemi identici**
- Memorizzare la soluzione di ciascun sottoproblema la prima volta che viene risolto, e riutilizzarla in seguito, è l'idea chiave della tecnica chiamata **Programmazione Dinamica**

Esempio: esecuzione per $n = 4$ (1/3)

Esecuzione di `Fib(4)`: le chiamate seguono l'ordine della visita in profondità dell'albero di `Fib(4)` visto sopra (indentazione = profondità nello stack).

chiamata	azione
`Fib(4)`	richiama `Fib(3)`
`  Fib(3)`	richiama `Fib(2)`
`    Fib(2)`	richiama `Fib(1)`
`      Fib(1)`	caso base, restituisce 1
`    Fib(2)`	richiama `Fib(0)`
`      Fib(0)`	caso base, restituisce 0
`    Fib(2)`	restituisce $1 + 0 = 1$
`  Fib(3)`	richiama `Fib(1)`
`    Fib(1)`	caso base, restituisce 1
`  Fib(3)`	restituisce $1 + 1 = 2$

Esempio: esecuzione per $n = 4$ (2/3)

L'esecuzione prosegue con la seconda chiamata ricorsiva di `Fib(4)`.

chiamata	azione
`Fib(4)`	richiama `Fib(2)`
`Fib(2)`	richiama `Fib(1)`
`Fib(1)`	caso base, restituisce 1
`Fib(2)`	richiama `Fib(0)`
`Fib(0)`	caso base, restituisce 0
`Fib(2)`	restituisce $1 + 0 = 1$
`Fib(4)`	restituisce $2 + 1 = 3$

Il risultato è $F(4) = 3$, dopo $C(4) = 2F(5) - 1 = 9$ chiamate.

Esempio: esecuzione per $n = 4$ (3/3)

`IterativeFib(4)`, invece, esegue 3 iterazioni:

iterazione	i	$next = prev + curr$	$prev$	$curr$
inizializzazione	2	–	0	1
1	$2 \to 3$	$0 + 1 = 1$	1	1
2	$3 \to 4$	$1 + 1 = 2$	1	2
3	$4 \to 5$	$1 + 2 = 3$	2	3

Il ciclo termina con $i = 5 > 4$ e la funzione restituisce $curr = 3$.

Riepilogo (1/2)

L'algoritmo ingenuo Fib:

- due casi base e due chiamate ricorsive; corretto per induzione forte
- l'albero delle chiamate ricorsive si ramifica: $C(n) = 2F(n+1) - 1$ chiamate

Crescita esponenziale:

- $2^{n/2} \leq C(n) \leq 2^n$ per $n \geq 2$
- $S(n) = 2C(n) = 4F(n+1) - 2 = 2^{\Theta(n)}$

Riepilogo (2/2)

L'algoritmo iterativo IterativeFib:

- calcola ciascun numero di Fibonacci una sola volta, conservando gli ultimi due valori
- $S(n) = 5n + 1 = \Theta(n)$

La lezione:

- il problema non è la ricorsione, ma la ripetizione di sottoproblemi identici
- memorizzarne e riutilizzarne le soluzioni è l'idea della Programmazione Dinamica

Massimo comun divisore

Introduzione: Massimo comun divisore

- In questa sezione riscriviamo come algoritmo ricorsivo l'algoritmo di Euclide, già visto nella sua forma iterativa
- È un esempio di **ricorsione in coda**: la chiamata ricorsiva è l'ultima operazione, e l'algoritmo si può riscrivere direttamente come un ciclo
- Dimostriamo che il secondo argomento si dimezza ogni due chiamate, il che dà un numero logaritmico di chiamate

Il problema

Dati due numeri naturali n, m con $n \geq 1$ e $m \geq 0$,
consideriamo il problema di calcolare
il loro massimo comun divisore $\gcd(n, m)$
(con $\gcd(n, 0) = n$).

- Nel capitolo «Algoritmi numerici» abbiamo dimostrato che, per tutti i numeri naturali $n \geq 1$ e $m \geq 1$,

$$\gcd(n, m) = \gcd(m,\ n \bmod m)$$

- e abbiamo ottenuto l'algoritmo iterativo `EuclideanAlgorithm`: esso sostituisce la coppia (n, m) con la coppia $(m,\ n \bmod m)$ più e più volte, finché $m = 0$

Una definizione ricorsiva

- La stessa proprietà, insieme a $\gcd(n, 0) = n$, è una definizione ricorsiva:

$$\gcd(n, m) = \begin{cases} n & \text{se } m = 0 \\ \gcd(m,\ n \bmod m) & \text{se } m \geq 1 \end{cases}$$

- Questo dà la versione ricorsiva dell'algoritmo di Euclide

Algorithm: RecursiveGCD

Input : Due numeri naturali $n \geq 1$ e $m \geq 0$
Output : $\gcd(n, m)$

```
1 function RecursiveGCD(n, m)
2     if m = 0 then
3         return n                                  // caso base
4     end
5     return RecursiveGCD(m, n mod m)               // passo
      ricorsivo
6 end
```

Esegui in Colab: Python · C++ · Java

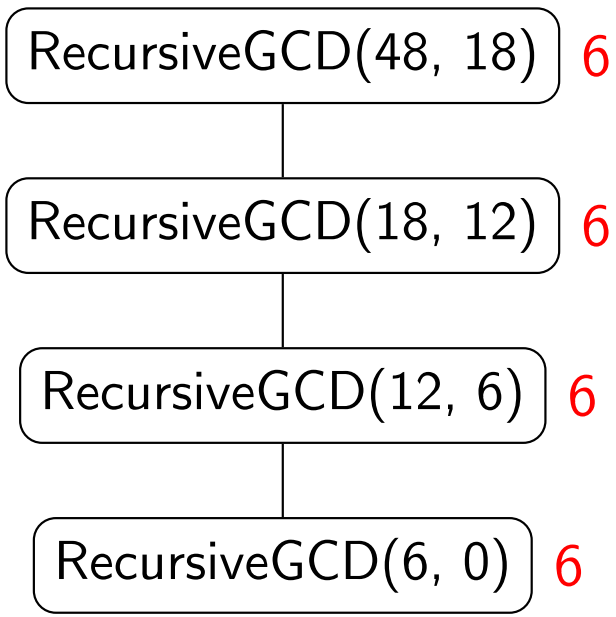
Terminazione di RecursiveGCD

La dimensione dell'input è il secondo argomento m.

Poiché $0 \leq n \bmod m < m$,
il secondo argomento diminuisce strettamente
a ogni chiamata ricorsiva,
quindi il caso base $m = 0$ viene sempre raggiunto.

Esempio: alberi delle chiamate di RecursiveGCD (1/2)

Ogni chiamata effettua al più una chiamata ricorsiva, quindi gli alberi sono cammini.





Esempio: alberi delle chiamate di RecursiveGCD (2/2)

- $48 \bmod 18 = 12$, $18 \bmod 12 = 6$, $12 \bmod 6 = 0$: quindi $\gcd(48, 18) = 6$
- $252 \bmod 105 = 42$, $105 \bmod 42 = 21$, $42 \bmod 21 = 0$: quindi $\gcd(252, 105) = 21$
- Il valore trovato dal caso base viene restituito invariato da tutte le chiamate

Osservazione 6: correttezza di RecursiveGCD

Per ogni coppia di numeri naturali $n \geq 1$ *e* $m \geq 0$, *la chiamata* `RecursiveGCD`(n, m) *termina e restituisce* $\gcd(n, m)$.

Dimostrazione

Procediamo per induzione forte su m.

- **Caso base** ($m = 0$): l'algoritmo restituisce $n = \gcd(n, 0)$ alla riga 3
- **Passo induttivo** ($m \geq 1$): supponiamo l'affermazione vera per ogni secondo argomento $k < m$ e per ogni primo argomento ≥ 1
  - l'algoritmo richiama `RecursiveGCD`$(m,\ n \bmod m)$, dove $m \geq 1$ e $n \bmod m < m$;
  - per ipotesi induttiva, questa chiamata restituisce $\gcd(m,\ n \bmod m)$;
  - questo è uguale a $\gcd(n, m)$ per la proprietà $\gcd(n, m) = \gcd(m,\ n \bmod m)$ richiamata all'inizio di questa sezione.

□

Definizione 4: ricorsione in coda

*Una chiamata ricorsiva è una **chiamata in coda** (**tail call**) se è l'ultima operazione eseguita dalla funzione chiamante e il suo risultato viene restituito invariato. Un algoritmo ricorsivo le cui chiamate ricorsive sono tutte chiamate in coda è **ricorsivo in coda**.*

Ricorsivo in coda oppure no?

- `RecursiveGCD` è ricorsivo in coda: la riga 5 restituisce direttamente il risultato della chiamata ricorsiva, senza alcuna ulteriore operazione

- `Factorial` **non** è ricorsivo in coda:
  - dopo che la chiamata ricorsiva `Factorial`$(n-1)$ è terminata, il risultato deve ancora essere moltiplicato per n;
  - quindi il frame del chiamante deve essere conservato nella pila.

Dalla ricorsione in coda a un ciclo

- Un algoritmo ricorsivo in coda può sempre essere trasformato in un ciclo `while`:
  - la chiamata in coda viene sostituita dall'aggiornamento dei parametri;
  - il ciclo continua finché non si raggiunge il caso base.

- Applicando questa trasformazione a `RecursiveGCD`, con l'aggiornamento

$$(n, m) \leftarrow (m,\ \ n \bmod m),$$

  otteniamo esattamente l'`EuclideanAlgorithm` iterativo, che usa una quantità costante di memoria

- Molti compilatori applicano automaticamente questa trasformazione (**tail-call optimization**)

Numero di passi: i secondi argomenti

- Ogni chiamata esegue esattamente 2 righe (le righe 2–3, oppure le righe 2 e 5), quindi il numero di passi è il doppio del numero di chiamate
- Limitiamo il numero di chiamate
- Sia $m_0 = m$ e siano $m_0, m_1, m_2, \ldots, m_K = 0$ i secondi argomenti delle chiamate successive
- La chiamata $k \geq 1$ è `RecursiveGCD`(m_{k-1}, m_k) e quindi

$$m_{k+1} = m_{k-1} \bmod m_k, \qquad m_{k+1} < m_k.$$

Osservazione 7: il secondo argomento si dimezza ogni due chiamate

Per ogni k tale che m_{k+2} sia definito,

$$m_{k+2} < \frac{m_k}{2}.$$

Dimostrazione

Abbiamo $m_{k+2} = m_k \bmod m_{k+1}$ con $m_{k+1} < m_k$. Distinguiamo due casi.

- Se $m_{k+1} \leq m_k/2$, allora

$$m_{k+2} < m_{k+1} \leq m_k/2.$$

- Se $m_{k+1} > m_k/2$, allora $m_k < 2m_{k+1}$, quindi il quoziente della divisione di m_k per m_{k+1} è 1 e

$$m_{k+2} = m_k - m_{k+1} < m_k - m_k/2 = m_k/2.$$

□

Limitare il numero di chiamate

- Sia $m \geq 1$ e $j = \lfloor \log_2 m \rfloor + 1$, in modo che $2^j > m$
- Applicando l'Osservazione j volte, se m_{2^j} è definito allora

$$m_{2^j} < \frac{m}{2^j} < 1 \quad \Longrightarrow \quad m_{2^j} = 0.$$

- Quindi $K \leq 2j$ e il numero di chiamate $(K + 1)$ è al più $2\lfloor \log_2 m \rfloor + 3$
- Perciò

$$S(n, m) \leq 2\big(2\lfloor \log_2 m \rfloor + 3\big) = O(\log_2 m).$$

Tempo di esecuzione di RecursiveGCD

- L'input è costituito dai due numeri n e m, quindi $|I| = \Omega(\log_2 m)$ (per $m \geq 2$) e, come per `ChangeToBase2` nel capitolo «Tempo di esecuzione», $S(n, m) = O(|I|)$

- L'algoritmo `RecursiveGCD` effettua al più $2\lfloor \log_2 m \rfloor + 3$ chiamate, e il suo tempo di esecuzione nel caso peggiore è

$$T(|I|) = O(|I|),$$

  cioè è al più **lineare** nella dimensione dell'input

- (Si può dimostrare che il caso peggiore si verifica per due numeri di Fibonacci consecutivi: per esempio $\gcd(89, 55)$ richiede 10 chiamate, contro il limite $2\lfloor \log_2 55 \rfloor + 3 = 13$.)

Esempio: esecuzione di RecursiveGCD(18, 48)

Per $n = 18$ e $m = 48$ (si noti che $n < m$: la prima chiamata scambia semplicemente i due numeri, poiché 18 mod 48 = 18):

1. `RecursiveGCD(18, 48)`: $m \neq 0$, 18 mod 48 = 18 $\Rightarrow$ richiama `RecursiveGCD(48, 18)`
2. `RecursiveGCD(48, 18)`: $m \neq 0$, 48 mod 18 = 12 $\Rightarrow$ richiama `RecursiveGCD(18, 12)`
3. `RecursiveGCD(18, 12)`: $m \neq 0$, 18 mod 12 = 6 $\Rightarrow$ richiama `RecursiveGCD(12, 6)`
4. `RecursiveGCD(12, 6)`: $m \neq 0$, 12 mod 6 = 0 $\Rightarrow$ richiama `RecursiveGCD(6, 0)`
5. `RecursiveGCD(6, 0)`: $m = 0 \Rightarrow$ caso base, restituisce 6

Il valore 6 viene restituito invariato da tutte le chiamate: $\gcd(18, 48) = 6$, dopo 5 chiamate.

Riepilogo

L'algoritmo:

- caso base $m = 0$, passo ricorsivo `RecursiveGCD`$(m,\ n \bmod m)$
- corretto per induzione forte sul secondo argomento m

Ricorsione in coda:

- la chiamata ricorsiva è l'ultima operazione e il suo risultato viene restituito invariato
- si può riscrivere come un ciclo: l'`EuclideanAlgorithm` iterativo

Numero di chiamate:

- il secondo argomento si dimezza ogni due chiamate: al più $2\lfloor \log_2 m \rfloor + 3$ chiamate
- $T(|I|) = O(|I|)$

Elevamento a potenza veloce

Introduzione: Elevamento a potenza veloce

- In questa sezione calcoliamo la potenza x^n mediante **elevamento al quadrato**: l'esponente si dimezza a ogni chiamata ricorsiva
- Dimostriamo la correttezza per induzione forte, usata per $\lfloor n/2 \rfloor$ invece che per $n-1$
- Srotolando l'equazione di ricorrenza $S(n) = S(\lfloor n/2 \rfloor) + 4$ otteniamo un numero **logaritmico** di passi, invece del numero lineare dell'algoritmo ingenuo

Il problema

Dati un numero x e un numero naturale n,
consideriamo il problema di calcolare la potenza x^n
(con $x^0 = 1$).

- L'algoritmo ingenuo moltiplica n volte per x:

$$p := 1, \qquad p \leftarrow p \cdot x \quad (n \text{ volte}),$$

che richiede n moltiplicazioni e $\Theta(n)$ passi

Elevamento al quadrato

- Un algoritmo molto più veloce si ottiene mediante **elevamento al quadrato**
- Poiché $n = 2\lfloor n/2 \rfloor$ se n è pari, e $n = 2\lfloor n/2 \rfloor + 1$ se n è dispari,

$$x^n = \begin{cases} 1 & \text{se } n = 0 \\ \left(x^{\lfloor n/2 \rfloor}\right)^2 & \text{se } n \geq 1 \text{ è pari} \\ x \cdot \left(x^{\lfloor n/2 \rfloor}\right)^2 & \text{se } n \text{ è dispari} \end{cases}$$

(per n dispari, $\lfloor n/2 \rfloor = (n-1)/2$)

- La dimensione dell'esponente si dimezza a ogni chiamata ricorsiva

Algorithm: FastPower

Input : Un numero x e un numero naturale n
Output : La potenza x^n

```
1  function FastPower(x, n)
2      if n = 0 then
3          return 1                                  // caso base
4      end
5      y := FastPower(x, ⌊n/2⌋)  // una chiamata ricorsiva
6      if n mod 2 = 0 then
7          return y · y                               // n pari
8      else
9          return x · y · y                          // n dispari
10     end
11 end
```

Esegui in Colab: Python · C++ · Java

Una chiamata ricorsiva, usata due volte

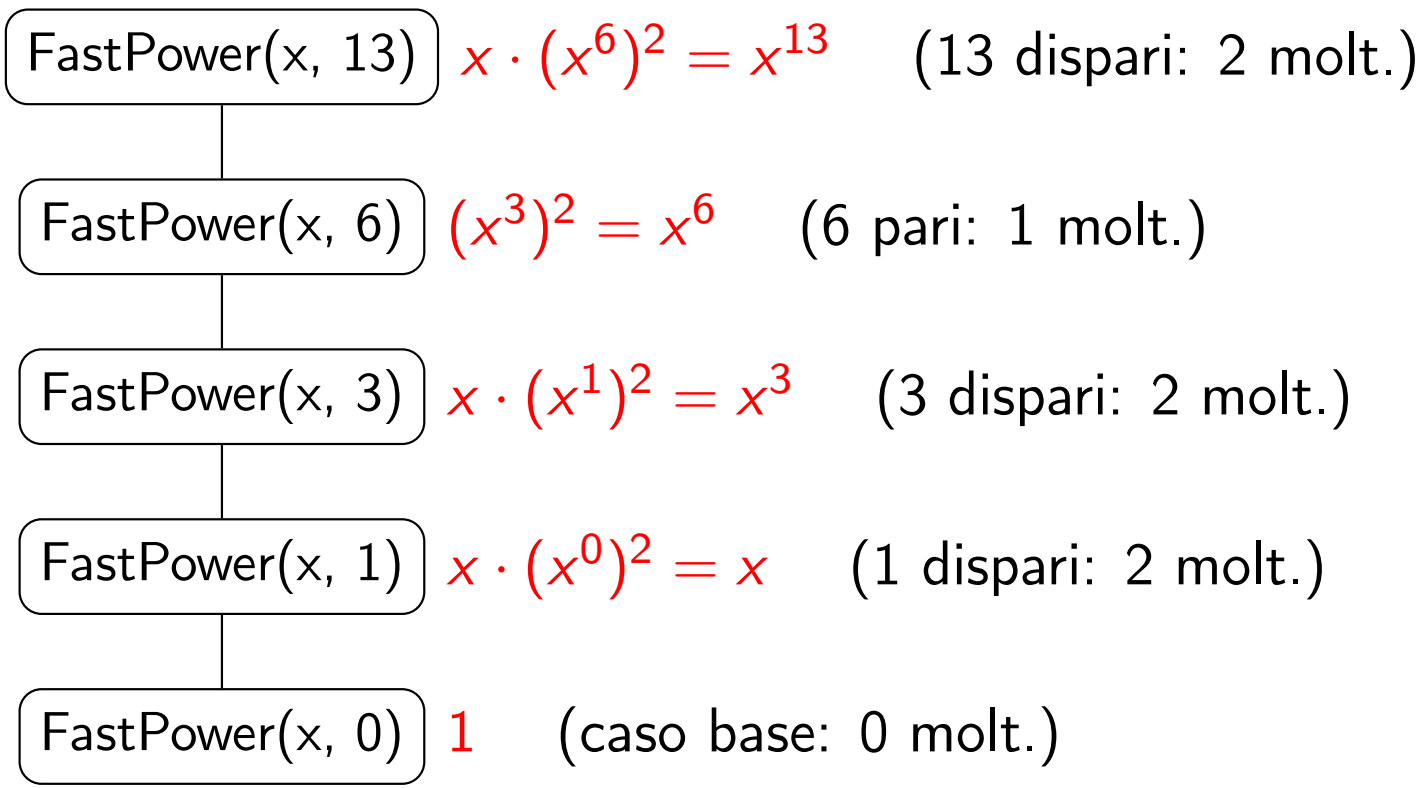
- Il risultato della chiamata ricorsiva è memorizzato nella variabile y e usato due volte
- Scrivere

$$\texttt{FastPower}(x, \lfloor n/2 \rfloor) \cdot \texttt{FastPower}(x, \lfloor n/2 \rfloor)$$

  produrrebbe **due** chiamate ricorsive identiche e distruggerebbe l'efficienza dell'algoritmo
- Questa variante è analizzata negli esercizi del capitolo

Esempio: albero delle chiamate di FastPower(x, 13) (1/2)

Gli esponenti delle chiamate sono $13, 6, 3, 1, 0$ ($\lfloor 13/2 \rfloor = 6$, $\lfloor 6/2 \rfloor = 3$, $\lfloor 3/2 \rfloor = 1$, $\lfloor 1/2 \rfloor = 0$). A destra di ciascuna chiamata: il valore restituito e il numero di moltiplicazioni eseguite da quella chiamata.



Esempio: albero delle chiamate di FastPower(x, 13) (2/2)

- Numero totale di moltiplicazioni: $2 + 1 + 2 + 2 + 0 = 7$, contro le 13 moltiplicazioni dell'algoritmo ingenuo
- Si noti che $13 = (1101)_2$:
  - gli esponenti dispari/pari incontrati salendo da $\texttt{FastPower}(x, 1)$ a $\texttt{FastPower}(x, 13)$ sono dispari, dispari, pari, dispari;
  - sono esattamente le cifre binarie $1, 1, 0, 1$ di 13 lette a partire dalla più significativa.
- In generale, il numero di moltiplicazioni è il numero di cifre binarie di n più il numero di cifre uguali a 1: qui $4 + 3 = 7$

Osservazione 8: correttezza di FastPower

Per ogni numero x e ogni numero naturale n, la chiamata `FastPower`(x, n) *termina e restituisce* x^n.

Dimostrazione

Procediamo per induzione forte su n.

- **Caso base** ($n = 0$): l'algoritmo restituisce $1 = x^0$ alla riga 3
- **Passo induttivo** ($n \geq 1$): supponiamo che `FastPower`(x, k) restituisca x^k per ogni $k < n$. Poiché $n \geq 1$, si ha $\lfloor n/2 \rfloor < n$, quindi per ipotesi induttiva la riga 5 assegna $y = x^{\lfloor n/2 \rfloor}$
  - se n è pari, allora $n = 2\lfloor n/2 \rfloor$ e la riga 7 restituisce $y \cdot y = x^{2\lfloor n/2 \rfloor} = x^n$;
  - se n è dispari, allora $n = 2\lfloor n/2 \rfloor + 1$ e la riga 9 restituisce $x \cdot y \cdot y = x^{2\lfloor n/2 \rfloor + 1} = x^n$.

Si noti che serve l'induzione forte: l'ipotesi induttiva viene usata per $k = \lfloor n/2 \rfloor$, non per $k = n - 1$. □

Numero di passi: l'equazione di ricorrenza

- Sia $S(n)$ il numero di passi di `FastPower` sull'esponente n
- Se $n = 0$, vengono eseguite le righe 2–3:

$$S(0) = 2.$$

- Se $n \geq 1$, le righe 2, 5, 6 e una delle righe 7, 9 vengono eseguite una volta ciascuna, più i passi della chiamata su $\lfloor n/2 \rfloor$:

$$S(n) = S(\lfloor n/2 \rfloor) + 4 \qquad (n \geq 1).$$

Numero di passi: srotolare l'equazione di ricorrenza (1/2)

Risolviamo l'equazione di ricorrenza srotolandola. Poiché $\lfloor \lfloor n/2^k \rfloor /2 \rfloor = \lfloor n/2^{k+1} \rfloor$,

$$\begin{aligned} S(n) &= S(\lfloor n/2 \rfloor) + 4 \\ &= S(\lfloor n/4 \rfloor) + 2 \cdot 4 \\ &\vdots \\ &= S(\lfloor n/2^k \rfloor) + k \cdot 4, \end{aligned}$$

fintanto che $\lfloor n/2^{k-1} \rfloor \geq 1$.

Numero di passi: srotolare l'equazione di ricorrenza (2/2)

- Ora, $\lfloor n/2^k \rfloor \geq 1$ se e solo se $2^k \leq n$, cioè $k \leq \lfloor \log_2 n \rfloor$
- Quindi, per $n \geq 1$, gli esponenti

$$n, \lfloor n/2 \rfloor, \ldots, \lfloor n/2^{\lfloor \log_2 n \rfloor} \rfloor = 1$$

  sono tutti ≥ 1, e il successivo è 0

- Scegliendo $k = \lfloor \log_2 n \rfloor + 1$:

$$S(n) = S(0) + 4\big(\lfloor \log_2 n \rfloor + 1\big) = 4\lfloor \log_2 n \rfloor + 6.$$

Passi, chiamate e moltiplicazioni

- Per esempio, $S(13) = 4 \cdot 3 + 6 = 18$
- Come mostrato per `ChangeToBase2` nel capitolo «Tempo di esecuzione»,

$$\lim_{n \to \infty} \frac{4\lfloor \log_2 n \rfloor + 6}{\log_2 n} = 4 \in \mathbb{R}_{>0},$$

  quindi $S(n) = \Theta(\log_2 n)$
- Per $n \geq 1$, il numero di chiamate, e quindi la profondità della ricorsione, è $\lfloor \log_2 n \rfloor + 2$ (il numero di cifre binarie di n, più la chiamata con esponente 0)
- Il numero di moltiplicazioni è al più $2(\lfloor \log_2 n \rfloor + 1)$, contro le n dell'algoritmo ingenuo
- Per esempio, per $n = 1000 = (1111101000)_2$: $10 + 6 = 16$ moltiplicazioni invece di 1000

Tempo di esecuzione di FastPower

- Supponiamo che x e tutti i risultati intermedi usino un numero costante di bit (così ogni moltiplicazione è un passo)
- La dimensione dell'input è $|I| = \Theta(\log_2 n)$, quindi, come per `ChangeToBase2` nel capitolo «Tempo di esecuzione», da $S(n) = \Theta(\log_2 n)$ segue $T(|I|) = \Theta(|I|)$

Il numero di passi dell'algoritmo `FastPower` è

$$S(n) = 4\lfloor \log_2 n \rfloor + 6 = \Theta(\log_2 n),$$

e il suo tempo di esecuzione nel caso peggiore è

$$T(|I|) = \Theta(|I|),$$

cioè è **lineare** nella dimensione dell'input. L'algoritmo ingenuo esegue $\Theta(n)$ passi, cioè è **esponenziale** nella dimensione dell'input.

Esempio: esecuzione di FastPower(3, 13) (1/2)

Per $x = 3$ e $n = 13$.

Chiamate

1. `FastPower(3, 13)`: $13 \neq 0 \Rightarrow$ richiama `FastPower(3, 6)` ($\lfloor 13/2 \rfloor = 6$)
2. `FastPower(3, 6)`: $6 \neq 0 \Rightarrow$ richiama `FastPower(3, 3)` ($\lfloor 6/2 \rfloor = 3$)
3. `FastPower(3, 3)`: $3 \neq 0 \Rightarrow$ richiama `FastPower(3, 1)` ($\lfloor 3/2 \rfloor = 1$)
4. `FastPower(3, 1)`: $1 \neq 0 \Rightarrow$ richiama `FastPower(3, 0)` ($\lfloor 1/2 \rfloor = 0$)
5. `FastPower(3, 0)`: caso base, restituisce 1

Esempio: esecuzione di FastPower(3, 13) (2/2)

Ritorni

1. `FastPower(3, 1)`: $y = 1$, 1 è dispari $\Rightarrow$ restituisce $3 \cdot 1 \cdot 1 = 3$
2. `FastPower(3, 3)`: $y = 3$, 3 è dispari $\Rightarrow$ restituisce $3 \cdot 3 \cdot 3 = 27$
3. `FastPower(3, 6)`: $y = 27$, 6 è pari $\Rightarrow$ restituisce $27 \cdot 27 = 729$
4. `FastPower(3, 13)`: $y = 729$, 13 è dispari $\Rightarrow$ restituisce $3 \cdot 729 \cdot 729 = 1594323$

L'algoritmo termina con risultato $1594323 = 3^{13}$, dopo 5 chiamate e 7 moltiplicazioni.

Riepilogo

L'algoritmo:

- una chiamata ricorsiva su $\lfloor n/2 \rfloor$, il cui risultato y è usato due volte
- corretto per induzione forte (ipotesi usata per $\lfloor n/2 \rfloor$)

Numero di passi:

- $S(n) = S(\lfloor n/2 \rfloor) + 4$, quindi $S(n) = 4\lfloor \log_2 n \rfloor + 6 = \Theta(\log_2 n)$
- profondità $\lfloor \log_2 n \rfloor + 2$; al più $2(\lfloor \log_2 n \rfloor + 1)$ moltiplicazioni

Tempo di esecuzione:

- $T(|I|) = \Theta(|I|)$, contro l'algoritmo ingenuo esponenziale

Torri di Hanoi

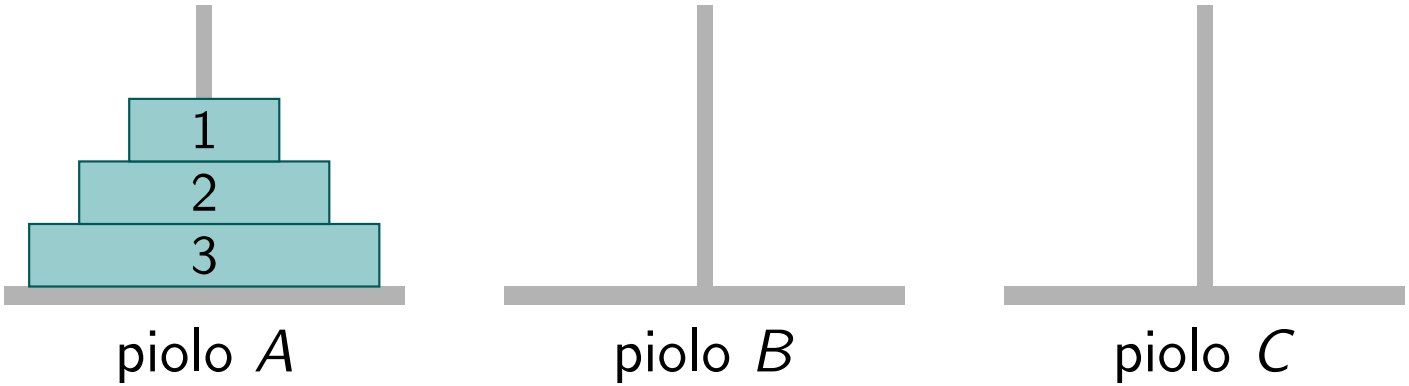
Introduzione: Torri di Hanoi

- In questa sezione risolviamo un rompicapo classico con un algoritmo ricorsivo che effettua **due** chiamate ricorsive su $n - 1$ dischi
- Ne disegniamo l'albero delle chiamate ricorsive e dimostriamo che esegue $2^n - 1$ mosse
- Infine mostriamo che nessun algoritmo può fare di meglio: il numero esponenziale di mosse è **ottimo**

Il problema

- Ci sono tre pioli A, B, C e $n \geq 1$ dischi di dimensioni diverse, numerati $1, 2, \ldots, n$ dal più piccolo al più grande
- Inizialmente tutti i dischi sono impilati sul piolo A, in ordine decrescente di dimensione dal basso verso l'alto
- Consideriamo il problema di spostare tutti i dischi sul piolo C, con le regole seguenti:
  - si sposta un solo disco alla volta, prendendo il disco in cima a un piolo e posandolo in cima a un altro piolo;
  - un disco non può mai essere posato sopra un disco più piccolo.

La configurazione iniziale



Tutti i dischi vanno spostati dal piolo A al piolo C,
uno alla volta, senza mai posare un disco sopra uno più piccolo.

Idea ricorsiva

Per spostare n dischi dal piolo A al piolo C usando il piolo B come piolo ausiliario:

1. spostare gli $n-1$ dischi più piccoli da A a B (usando C come piolo ausiliario), ricorsivamente;
2. spostare il disco più grande n da A a C;
3. spostare gli $n-1$ dischi più piccoli da B a C (usando A come piolo ausiliario), ricorsivamente.

- Durante i passi 1 e 3, il disco n si trova in fondo a un piolo ed è più grande di tutti i dischi spostati, quindi non viola mai le regole
- Gli $n-1$ dischi possono essere spostati come se il disco n non ci fosse

Algorithm: Hanoi

Input : Il numero di dischi $n \geq 1$, il piolo di partenza *from*, il piolo di arrivo *to*, il piolo ausiliario *aux*

Output : La sequenza di mosse che trasferisce i dischi $1, 2, \ldots, n$ da *from* a *to*

```
1 function Hanoi(n, from, to, aux)
2     if n = 1 then
3         sposta il disco 1 da from a to          // caso base
4     else
5         Hanoi(n − 1, from, aux, to)
6         sposta il disco n da from a to
7         Hanoi(n − 1, aux, to, from)
8     end
9 end
```

La chiamata iniziale è `Hanoi`(n, A, C, B).

Esegui in Colab: Python · C++ · Java

Osservazione 9: correttezza di Hanoi

Per ogni numero naturale $n \geq 1$*, se i dischi* $1, 2, \ldots, n$ *sono gli* n *dischi in cima al piolo from e tutti i dischi sui pioli to e aux sono più grandi di* n*, allora* `Hanoi`$(n, from, to, aux)$ *sposta i dischi* $1, 2, \ldots, n$ *in cima al piolo to, rispettando le regole.*

Dimostrazione

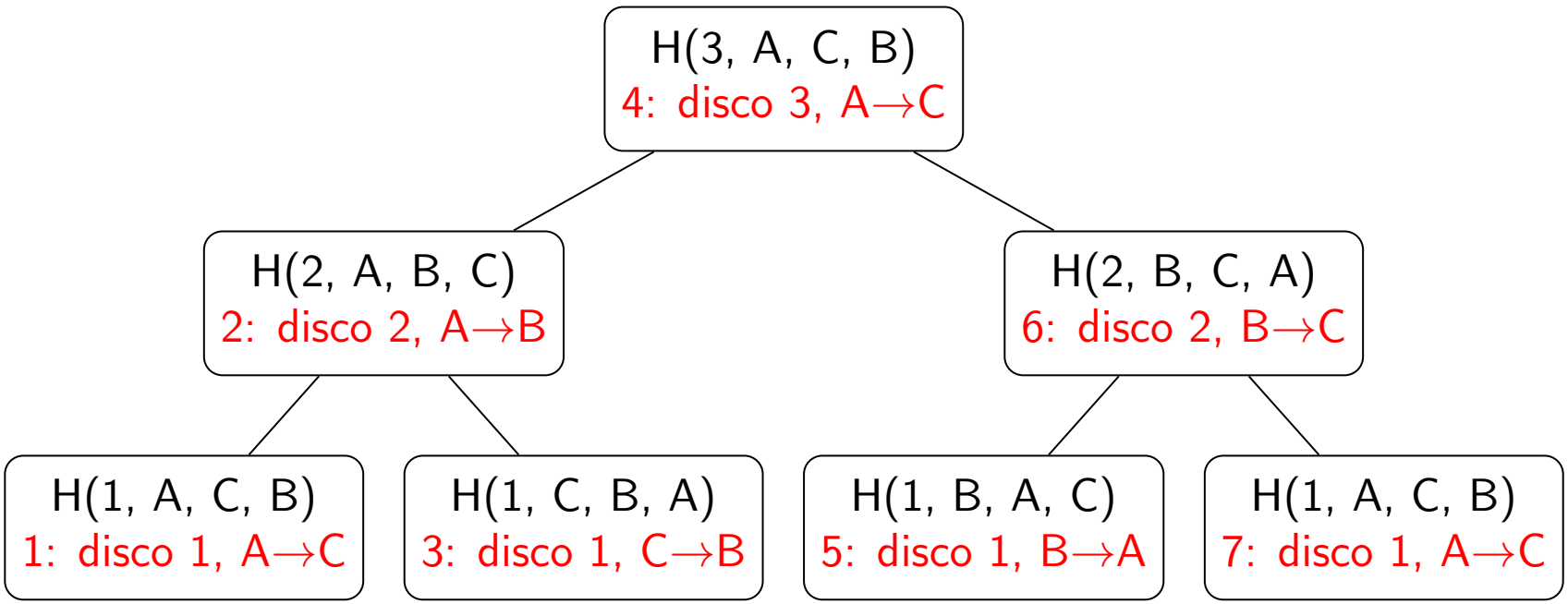
Procediamo per induzione su n.

- **Caso base** ($n = 1$): l'unica mossa del disco 1 è valida, poiché il disco 1 è il disco più piccolo
- **Passo induttivo** ($n \geq 2$):
  - per ipotesi induttiva, la chiamata della riga 5 sposta correttamente la torre degli $n - 1$ dischi più piccoli da *from* a *aux* (il disco n e i dischi sotto di esso sono più grandi, quindi non interferiscono);
  - allora il disco n è in cima al piolo *from*, e ogni disco sul piolo *to* è più grande di n: la riga 6 sposta correttamente il disco n;
  - infine, per ipotesi induttiva, la chiamata della riga 7 sposta gli $n - 1$ dischi più piccoli da *aux* in cima al disco n.

□

Esempio: albero delle chiamate di Hanoi(3, A, C, B) (1/2)

Ogni nodo mostra la chiamata (con argomenti *n*, *from*, *to*, *aux*) e, in rosso, la mossa eseguita dalla chiamata stessa (riga 3 o riga 6), numerata nell'ordine di esecuzione.



Esempio: albero delle chiamate di Hanoi(3, A, C, B) (2/2)

Le mosse vengono eseguite nell'ordine di una visita **simmetrica** (**in-order**) dell'albero (sottoalbero sinistro, nodo, sottoalbero destro):

mossa	disco	da	a
1	1	A	C
2	2	A	B
3	1	C	B
4	3	A	C
5	1	B	A
6	2	B	C
7	1	A	C

L'albero ha 7 nodi e ogni nodo esegue esattamente una mossa: 7 mosse in totale.

Numero di mosse

- Sia $M(n)$ il numero di mosse eseguite da `Hanoi` con n dischi. Dallo pseudocodice,

$$M(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2M(n-1) + 1 & \text{se } n \geq 2 \end{cases}$$

Numero di mosse: srotolamento

Srotolando:

$$\begin{aligned} M(n) &= 2M(n-1) + 1 \\ &= 4M(n-2) + 2 + 1 \\ &= 8M(n-3) + 4 + 2 + 1 \\ &\vdots \\ &= 2^{n-1}M(1) + 2^{n-2} + \cdots + 2 + 1 = 2^n - 1. \end{aligned}$$

Lo dimostriamo formalmente.

Osservazione 10: numero di mosse di Hanoi

Per ogni numero naturale $n \geq 1$,

$$M(n) = 2^n - 1.$$

Dimostrazione

Procediamo per induzione su n.

- **Caso base** ($n = 1$): $M(1) = 1 = 2^1 - 1$
- **Passo induttivo** ($n \geq 2$): supponiamo $M(n-1) = 2^{n-1} - 1$. Allora

$$\begin{aligned} M(n) &= 2M(n-1) + 1 \\ &= 2\left(2^{n-1} - 1\right) + 1 \\ &= 2^n - 2 + 1 = 2^n - 1. \end{aligned}$$

□

Numero di passi e profondità

- Il numero di passi soddisfa
  - $S(1) = 2$ (righe 2–3);
  - $S(n) = 2S(n-1) + 4$ per $n \geq 2$ (righe 2, 5, 6, 7, più le due chiamate ricorsive).
- Srotolando come sopra,

$$\begin{aligned} S(n) &= 2^{n-1}S(1) + 4\left(2^{n-2} + \cdots + 2 + 1\right) \\ &= 2^n + 4\left(2^{n-1} - 1\right) = 3 \cdot 2^n - 4 = \Theta(2^n), \end{aligned}$$

  poiché $\lim_{n\to\infty}(3 \cdot 2^n - 4)/2^n = 3 \in \mathbb{R}_{>0}$
- La profondità della ricorsione è solo n: la memoria usata dal call stack è $\Theta(n)$

Ottimalità

Il numero esponenziale di mosse
non è una debolezza dell'algoritmo:

nessun algoritmo può risolvere il problema
con meno mosse.

Osservazione 11: ottimalità di Hanoi

Sia $M^*(n)$ il minimo numero di mosse necessarie per trasferire una torre di $n \geq 1$ dischi da un piolo a un altro. Allora

$$M^*(n) = 2^n - 1.$$

Traccia della dimostrazione (1/3)

- Poiché l'algoritmo `Hanoi` usa $2^n - 1$ mosse, $M^*(n) \leq 2^n - 1$
- Dimostriamo $M^*(n) \geq 2^n - 1$ per induzione su n
- Per $n = 1$ serve almeno una mossa
- Sia $n \geq 2$. Il disco più grande n deve essere spostato **almeno una volta**

Traccia della dimostrazione (2/3)

- Consideriamo la **prima** mossa del disco n, dal piolo di partenza a un altro piolo P:
  - in quel momento il disco n è l'unico disco sul piolo di partenza e il piolo P è vuoto;
  - quindi gli $n-1$ dischi più piccoli sono tutti impilati sul terzo piolo;
  - spostarli lì dal piolo di partenza ha richiesto almeno $M^*(n-1)$ mosse.

- Consideriamo ora l'**ultima** mossa del disco n, verso il piolo di arrivo:
  - dopo questa mossa, gli $n-1$ dischi più piccoli sono tutti impilati su un piolo diverso da quello di arrivo;
  - devono ancora essere trasferiti in cima al disco n: ciò richiede almeno altre $M^*(n-1)$ mosse.

Traccia della dimostrazione (3/3)

Questi due gruppi di mosse sono distinti (uno avviene prima della prima mossa del disco n, l'altro dopo la sua ultima mossa), quindi

$$\begin{aligned} M^*(n) &\geq M^*(n-1) + 1 + M^*(n-1) \\ &= 2M^*(n-1) + 1 \\ &\geq 2\left(2^{n-1} - 1\right) + 1 = 2^n - 1. \end{aligned}$$

□

Tempo di esecuzione di Hanoi

- L'algoritmo `Hanoi` esegue $M(n) = 2^n - 1$ mosse e $S(n) = 3 \cdot 2^n - 4 = \Theta(2^n)$ passi, con profondità di ricorsione n
- Il numero di mosse è **ottimo**: ogni algoritmo deve produrre in output $2^n - 1$ mosse
- Quindi il tempo di esecuzione di qualunque algoritmo per questo problema è esponenziale in n (e quindi doppiamente esponenziale nella dimensione dell'input $|I| = \Theta(\log_2 n)$)
- Con $n = 64$ dischi (la leggenda originale) e una mossa al secondo, $2^{64} - 1$ secondi sono più di $5.8 \cdot 10^{11}$ anni

Esempio: esecuzione di Hanoi(3, A, C, B) (1/2)

Indentazione = profondità nello stack; le righe stampate sono le mosse dell'albero visto sopra.

chiamata	azione
`Hanoi(3,A,C,B)`	richiama `Hanoi(2,A,B,C)`
`  Hanoi(2,A,B,C)`	richiama `Hanoi(1,A,C,B)`
`    Hanoi(1,A,C,B)`	stampa **sposta il disco 1 da A a C**, termina
`  Hanoi(2,A,B,C)`	stampa **sposta il disco 2 da A a B**, richiama `Hanoi(1,C,B,A)`
`    Hanoi(1,C,B,A)`	stampa **sposta il disco 1 da C a B**, termina
`  Hanoi(2,A,B,C)`	termina
`Hanoi(3,A,C,B)`	stampa **sposta il disco 3 da A a C**, richiama `Hanoi(2,B,C,A)`

Esempio: esecuzione di Hanoi(3, A, C, B) (2/2)

chiamata	azione
`Hanoi(2,B,C,A)`	richiama `Hanoi(1,B,A,C)`
`  Hanoi(1,B,A,C)`	stampa **sposta il disco 1 da B a A**, termina
`Hanoi(2,B,C,A)`	stampa **sposta il disco 2 da B a C**, richiama `Hanoi(1,A,C,B)`
`  Hanoi(1,A,C,B)`	stampa **sposta il disco 1 da A a C**, termina
`Hanoi(2,B,C,A)`	termina
`Hanoi(3,A,C,B)`	termina

Tutte le 7 mosse vengono stampate e tutti i dischi sono sul piolo C.

Riepilogo

L'algoritmo:

- sposta $n-1$ dischi sul piolo ausiliario, sposta il disco n, riporta gli $n-1$ dischi sopra di esso
- corretto per induzione su n; le mosse seguono una visita simmetrica dell'albero

Numero di mosse e di passi:

- $M(n) = 2M(n-1)+1 = 2^n - 1$; $S(n) = 3 \cdot 2^n - 4 = \Theta(2^n)$; profondità n

Ottimalità:

- ogni algoritmo richiede almeno $2^n - 1$ mosse: è il problema stesso a essere esponenziale

Riepilogo: i cinque algoritmi (1/3)

algoritmo	equazione di ricorrenza per $S(n)$
`Factorial`	$S(n) = S(n-1) + 2$
`Fib` (ingenuo)	$S(n) = S(n-1) + S(n-2) + 2$
`RecursiveGCD`	$m_{k+2} < m_k/2$
`FastPower`	$S(n) = S(\lfloor n/2 \rfloor) + 4$
`Hanoi`	$S(n) = 2S(n-1) + 4$

Riepilogo: i cinque algoritmi (2/3)

algoritmo	$S(n)$
`Factorial`	$2n + 2 = \Theta(n)$
`Fib` (ingenuo)	$4F(n+1) - 2 = 2^{\Theta(n)}$
`RecursiveGCD`	$O(\log_2 m)$
`FastPower`	$4\lfloor \log_2 n \rfloor + 6 = \Theta(\log_2 n)$
`Hanoi`	$3 \cdot 2^n - 4 = \Theta(2^n)$

Riepilogo: i cinque algoritmi (3/3)

algoritmo	chiamate	profondità
`Factorial`	$n+1$	$n+1$
`Fib` (ingenuo)	$2F(n+1)-1$	$\max(n,1)$
`RecursiveGCD`	$\leq 2\lfloor \log_2 m \rfloor + 3$	$=$ chiamate
`FastPower`	$\lfloor \log_2 n \rfloor + 2$	$=$ chiamate
`Hanoi`	$2^n - 1$	n

Riepilogo: i punti chiave del capitolo (1/2)

- Un **algoritmo ricorsivo** risolve direttamente i **casi base** e riduce ogni altra istanza a istanze **strettamente più piccole** dello stesso problema; questo garantisce la terminazione
- L'esecuzione è descritta dall'**albero delle chiamate ricorsive**: il numero di passi dipende dal **numero di nodi**, la memoria usata dal **call stack** dalla **profondità** dell'albero
- La correttezza si dimostra per **induzione (forte)** sulla dimensione dell'input: casi base $\leftrightarrow$ base dell'induzione, passo ricorsivo $\leftrightarrow$ passo induttivo (come inizializzazione e conservazione per gli invarianti di ciclo)

Riepilogo: i punti chiave del capitolo (2/2)

- Il numero di passi è descritto da un'**equazione di ricorrenza** per $S(n)$:
  - una diminuzione di uno con lavoro costante dà $\Theta(n)$ (`Factorial`);
  - un dimezzamento con lavoro costante dà $\Theta(\log_2 n)$ (`FastPower`);
  - due chiamate su $n-1$ danno $\Theta(2^n)$ (`Hanoi`).
- La ricorsione può essere molto inefficiente quando gli **stessi sottoproblemi** vengono risolti molte volte (`Fib` ingenuo): questo motiva la **Programmazione Dinamica**
- Un algoritmo **ricorsivo in coda** (`RecursiveGCD`) può essere riscritto direttamente come un ciclo